

MEMORIA 1942

Grado universitario:

Ingeniería Informática(Gp 82. 2022-2023).

TRABAJO PROYECTO FINAL PROGRAMACIÓN.

ALUMNOS:

JORGE LÓPEZ ALONSO
CARLOS MORENO GONZÁLEZ

TUTORES:
ANA-MARÍA BLANCO BENITO
GUSTAVO FERNANDEZ-BAILLO CAÑAS

UC3M CAMPUS LEGANÉS

12-12-2022

ÍNDICE

1. Introducción.
2. Clase Avión.
3. Constantes.
4. Clase Islas.
5. Clase Enemigos.
 - 5.1. Clase Regular.
 - 5.2. Clase Rojos.
 - 5.3. Clase Bombardero.
 - 5.4. Clase superbombardero.
6. Clase Proyectoil.
7. Clase Misiles.
8. Clase Tablero.
9. Conclusión.

1. INTRODUCCIÓN

En este trabajo trataremos de explicarles con el mayor nivel de detalle posible el trabajo realizado para desarrollar esta réplica del videojuego retro “1942”.

Empezaremos hablándoles sobre las clases más sencillas, como puede ser la clase avión y el archivo donde guardaremos las constantes, seguiremos con la clase “enemigos” y sus diferentes subclases, como por ejemplo la clase regular, para terminar hablando de la clase más compleja, llamada “tablero”.

Esperamos que les guste el resultado del trabajo realizado por Carlos Moreno González y Jorge López Alonso. Dicha dupla, ha trabajado sin cesar y a contrarreloj para tratar lograr un resultado lo más parecido posible a la versión original de este juego.

2. CLASE AVIÓN

Esta clase es una de las más sencillas de todo el proyecto. Esto se debe a que solo consta de tres métodos, que son: el método init y el método mover.

Método init:

Requiere de los parámetros x e y, siendo estos la posición del avión; dos sprites, que simulan el movimiento del avión haciendo girar sus hélices, y las vidas de dicho avión(tres).

Método mover:

Dicho método permite al avión desplazarse en cualquier dirección del mapa sin salirse de la pantalla.

3. ARCHIVO.PY “CONSTANTES”

Este archivo lo utilizaremos simplemente a modo de banco de imágenes(sprites, islas, ancho y alto del tablero...)

4. CLASE ISLAS

En esta clase únicamente describimos el movimiento de las islas (x e y), y su representación gráfica en el mapa(sprites), dándole a cada isla un tipo diferente para que posteriormente se pinten como objetos independientes.

5. CLASE ENEMIGOS

Esta clase será la clase “madre”, por así decirlo, de las siguientes clases que comentaremos más tarde. Consta de 6 métodos: método init, método mover, y después cuatro métodos indicando las direcciones del método mover.

Método init:

Consta de los parámetros de posición x e y, y el tipo de enemigo, seguido de los atributos por defecto de dirección, velocidad y vidas.

Método mover:

Este método sirve para que los enemigos se muevan a una determinada velocidad y dirección.

Métodos de dirección:

Son unos métodos que llaman al método mover. Su función es facilitar la lectura y escritura del código.

5.1 CLASE REGULAR(ENEMIGOS)

Con un super init dentro del método init para seguir el ejemplo de la clase madre, sus atributos diferenciadores serán los debidos para especificar el tamaño del jugador(se utilizará en el siguiente método), el atributo volver(se utilizará en el siguiente método), y los atributos de velocidad, munición y tipo propios.

Método movimiento:

Recibirá tres parámetros: z(posición del regular) v y u(posiciones en el eje x de los dos aviones).

En este método se describe el movimiento de los aviones regulares. Estos aviones describen una trayectoria descendente en el eje Y, tratando de seguir al avión del jugador, hasta que alcanzan una posición aleatoria de retorno en la que dan la vuelta hasta desaparecer.

5.2 CLASE ROJOS(ENEMIGOS)

Dentro del método init será prácticamente idéntico al de la anterior clase, a excepción del atributo de velocidad, que será menor. El método movimiento hará que estos enemigos describan una trayectoria horizontal cuya dirección cambiará en un ángulo de 180 grados al llegar al borde de la pantalla.

5.3 CLASE BOMBARDERO(ENEMIGOS)

Como ya hemos comentado anteriormente, el método init será igual a los anteriores a excepción de un nuevo atributo llamado contador y de los atributos velocidad y munición. El método movimiento hará que el bombardero describa una trayectoria descendente hasta la posición $y = 150$,

donde comenzará a describir un cuadrado de 100 píxeles de lado y posteriormente desaparecerá del mapa por la parte inferior.

5.4 CLASE SUPERBOMBARDERO

(ENEMIGOS)

Como ya hemos comentado anteriormente, el método init será igual a las clases anteriores a excepción del atributo vuelta que ya vimos anteriormente en la clase regular. El método de movimiento de esta clase hace que el superbombardero, que aparece por la parte inferior de la pantalla, suba hasta una posición recibida por el parámetro y. Después de esto hará un movimiento similar al de los enemigos rojos.

6. CLASE PROYECTIL

La clase proyectil viene a describir la aparición y desplazamiento de los misiles lanzados por el avión. Encontramos dos métodos:

Método init:

Al igual que la mayoría del resto de inits descritos anteriormente, este nos describe los parámetros de posición x e y del misil, y el sprite(las coordenadas para sacar el grafismo del proyectil en cuestión).

Método Mover:

Este sencillo método simplemente nos da a conocer que el misil se desplaza únicamente en el eje y, en dirección ascendente.

7. CLASE MISILES

Los métodos `inits` y `mover` de los misiles enemigos son idénticos a los de la clase `Enemigos`, anteriormente vistos.

Lo importante de esta clase, es el método `movimiento`, pues sirve para que los misiles se muevan con función de autoguiado hacia el avión del jugador durante poco más de un segundo y después se muevan hacia abajo.

La clase `Misil SuperBombardero`, es una subclase de la anteriormente mencionada, pero no está del todo aprovechada.

8. CLASE TABLERO

Dentro del método `init` de esta clase una cantidad considerable de atributos que irán complementando a los métodos más adelante. Cabe destacar en este la inicialización de listas para misiles y enemigos, los atributos de puntuación y un bucle el cual inicializa las islas.

Métodos gráficos:

En ellos encontraremos los diferentes métodos que llamaremos posteriormente en el método `draw` para pintar los diferentes misiles, aviones, islas y elementos gráficos como la portada. Son métodos muy sencillos.

Métodos de control de apariciones:

Estos métodos son un poco más complejos que los anteriores. Se encargan de hacer aparecer a los enemigos progresivamente en un intervalo de tiempo determinado mediante el uso del comando de `pyxel.frame_count` y condicionales.

Métodos de Colisiones:

- Primer método: se encarga de las colisiones producidas entre los enemigos y los misiles lanzados por el avión del jugador. Si el proyectil impacta contra un enemigo, es decir, entra dentro de la hitbox de dicho enemigo, ambos irán a parar en un punto muy lejano en mapa, prácticamente en el infinito, y la puntuación subirá dependiendo del grado de dificultad del enemigo. Hay un bucle para cada tipo de enemigo.
- Segundo método: al igual que en el método anterior, este se encarga de las colisiones, pero esta vez entre los misiles de los enemigos y el avión del jugador. El funcionamiento es exactamente igual que en el primer método.
- Tercer método: este método es idéntico a los anteriores, pero se encarga de las colisiones entre el avión del jugador y las aeronaves enemigas. Repitiendo de nuevo el funcionamiento de los dos métodos anteriores.
- Cabe destacar que si el avión del jugador recibe una colisión perderá una vida, así sucesivamente hasta cumplir el cupo de 3 vidas, lo que cerrará el juego.

Método draw:

Este método imprime la portada hasta que la letra M sea presionada, empezando así el videojuego. Cabe destacar en el mismo, la utilización del comando `pyxel.text`, y el respeto de una estructura piramidal a la hora de la llamada de los métodos gráficos.

Método Update:

Este método actualiza constantemente la pantalla, permitiendo que los elementos se desplacen de forma fluida. Mediante bucles y condicionales nos permite mover el avión con las flechas del teclado y disparar con el espacio(todo esto utilizando el comando `pyxel.KEY_`). Además, se encarga de actualizar/disparar los misiles de las aeronaves enemigas y de mover a cada uno de estos. También se encarga de refrescar la puntuación.

9. CONCLUSIÓN

Para terminar este trabajo, queremos pedir disculpas por no haber conseguido una versión mejor. Creemos que hemos logrado un buen trabajo, pero nos hubiera gustado llegar hasta el final de este y añadirle detalles personales que lo hicieran único e inigualable. Desgraciadamente esto no va a poder ser, ya que el tiempo es limitado y a pesar de haber estado trabajando durante semanas, en especial la última semana, no hemos conseguido tanto como esperábamos. Consideramos que hubiésemos sido capaces de hacer aún más, pero estamos muy contentos de haber trabajado codo con codo, aprendiendo cada día infinidad de cosas sobre Python, y reforzando nuestra amistad.

El principal problema que hemos tenido ha sido desde un principio el desconocimiento de este nuevo lenguaje, teniendo que gastar bastante tiempo en buscar y entender el funcionamiento del código. Nos hubiera gustado una mayor profundización en el temario, no solo en las clases de teoría, sino también en las clases prácticas. Otros problemas que nos han hecho pararnos a pensar durante largos ratos, han sido las Hitbox de las colisiones, llegando a tener que dibujarnos las Hitbox en un paint para lograr entenderlas. Así como la utilización de los bancos de imágenes, haciendo equivocarnos creando 2 ficheros `pyxres` en vez de solo 1 por culpa del desconocimiento, haciendo la labor de cuadrar los sprites de cada elemento

con su fichero pyxres un calvario el cuál hemos solucionado poniendo el comando `pyxel.load` repetidas veces en los métodos gráficos.